

Documentation

The Process of Creating My Website “handgemacht von Svea”

Starting Point: Design Mockup & Color Scheme

The foundation was the design mockup from a seminar last year. Since I had already decided on the colors (yellow, cream, and taupe as an accent) and fonts early on, I was able to use them right away in the practical assignments. For example, in the Flex-Layout-assignment, which had already served as the basis for my homepage. So by the end of the classes, I had an initial prototype that I could build on.

Code Basics: Cleanup & Design Tokens

Before building any more pages, I cleaned up my code and introduced variables for colors and spacing. This decision paid off in ensuring a consistent design across all nine pages: changes to spacing or color shades could be made centrally, instead of having to go through each page individually. I finalized the navigation bar and footer so that they could be copied unchanged to any subsequent page and function identically everywhere.

Implementation of the Remaining Static Pages

The next step was implementing the remaining pages in a static version. The “Offers,” “Workshops,” “Gallery,” “About Me,” and “Contact” sections followed a similar layout. Therefore once I had my layout, I could focus on selecting appropriate images and text and ensuring that my brand's tone remained consistent across all pages.

Mobile Optimization & Responsive Bug Fixes

Over several iterations, I fixed bugs in the responsive design, such as a layout error with the hero image that created unexpected white space on mobile, or the image gallery, which needed to wrap cleanly on different screen sizes. I also consistently refined the mobile version across all pages. Furthermore, I revised the mobile navigation several times. I was significantly aided in this process by AI, which explained existing bugs to me and provided suggested solutions. It was important to me to understand every code suggestion before implementing it. The part of Bug Fixing Responsive and Mobile Version Bugs was the most time-consuming.

Accessibility & Search Engine Optimization

It was important to me that this website, which I actually publish is accessible. I implemented a skip link (“skip to content”), visible focus states when tapping through the page, lazy loading, and correct ARIA attributes on interactive elements as well as Alt-texts for images.

At the same time, I optimized the pages for search engines: page-specific meta keywords, Open Graph tags and revised meta titles and descriptions according to my keywords (“Handgemacht”, “Upcycling”, “Lüneburg” etc.). I reduced the image sizes in this step, but I would later realize that this was far from enough.

Gallery Features: Load More & PhotoSwipe

As the number of images grew, the gallery became crowded by images as well as too slow to load initially. With a “Load More” button, I now load only a portion of the images initially. The rest load

only when clicked. I also integrated PhotoSwipe so that images can be viewed in detail in a lightbox with zoom, swipe, and keyboard controls. I see this as an especially important feature to showcase my handmade jewelry.

User Testing

I asked a friend if she could test my website on usability. She provided great feedback, mainly on the content. So made some changes to my texts and I renamed the “Inspiration” page to “Gallery” to make the page’s purpose clear.

Performance Optimization: Images & Lighthouse

I found Lighthouse to be an extremely useful tool that clearly pointed out performance issues and even provided tips on how to fix them. The first Lighthouse test showed a score of 96 for Accessibility, 100 for Best Practices, and 100 for SEO, but only 72 for Performance (even though the images were already compressed). The causes were a lack of modern image formats (WebP) and oversized image dimensions. With the help of Claude, I automatically generated multiple versions of all 124 images (WebP + fallback, three sizes each) and replaced all tags with <picture> elements using srcset (which would have been impractical to do manually given the volume). The result is a reduction in data volume on the gallery page from ~40 MB to 6–10 MB and an overall Lighthouse Performance score of 99.

Deployment & Versioning via GitHub

After doing some research, I decided to host the site on Netlify so that it would look professional to visitors and I still have plenty of room to grow in the future, maybe even with my own domain (without Netlify in it). The GitHub repository is directly connected to the live deployment via Netlify, allowing me to publish changes easily and automatically.

Resources

Throughout the development process, I used some online resources to support my work. W3Schools was particularly helpful for looking up HTML, CSS, JavaScript, and responsive design concepts. I also referred to the official documentation of tools and libraries such as PhotoSwipe, Netlify, and the Lighthouse-tips whenever I needed more detailed information or guidance.

AI Use & Labeling

I used AI selectively and clearly labeled its use, particularly for JavaScript code, accessibility features, responsive bug fixes, and alt-text generation. In other words, I used it either to explain bugs and provide solutions, when the desired functionality was well beyond my knowledge or to speed up repetitive tasks. My guiding principle was always to understand and adjust any code I got by AI before using it, instead of adopting it without careful consideration.

Conclusion

From the first prototype created for the seminar assignments to the live website, it was an iterative process: first structure and design, then content, followed by responsive and mobile optimization, accessibility, SEO, and finally performance. In particular, responsive bug fixing and image optimization demonstrated just how much detailed work goes into a seemingly simple website and how helpful the targeted, transparent use of AI was in this process, without losing track of my own code.